Navigation

- System Hub
- ONYX — The Orchestration Layer for Obsidian

ONYX — The Orchestration Layer for Obsidian

> Turn your Obsidian vault into an autonomous execution surface. One nervous system. Many specializations. Every project — code, content, research, or operations — running on the same phase-driven lifecycle.

Mission Statement

ONYX exists to solve one problem: **knowledge work has no operating layer.**

You have project management tools (Jira, Linear, Notion). You have AI agents (Claude, Cursor, GPT). You have knowledge bases (Obsidian, wikis, docs). But nothing connects intent to execution to learning in a single, inspectable loop.

ONYX is that loop.

It treats your Obsidian vault as a **stateful execution surface** — not just a place to store notes, but the actual machine that coordinates what happens next. Phase notes carry state. Log notes carry evidence. Knowledge notes carry compounding wisdom. The controller reads vault state, dispatches agents, writes results back. Everything is markdown. Everything is inspectable. Everything is yours.

The mission: **make autonomous, multi-project, multi-domain execution reliable, observable, and compounding** — without requiring any infrastructure beyond an Obsidian vault and a CLI.

Philosophy

The Three Convictions

1. The vault is the only truth.
No external databases. No SaaS dependencies for state. No `.lock.json` files on disk. If a phase is active, the vault says so. If an agent produced output, the log note shows it. If a lesson

was learned, Knowledge.md holds it. The vault is not documentation about the work — it IS the work's state machine.

2. Execution must be domain-agnostic.
A phase is a phase. Whether you're shipping a TypeScript feature, producing a podcast episode, synthesizing a research paper, or running a trading strategy — the lifecycle is the same: plan, decompose, execute, verify, consolidate learnings. The *what* changes. The *how it's managed* doesn't.

3. Knowledge must compound.
Every completed phase produces learnings, decisions, and gotchas. These are written back into the vault where future phases — and future projects — can read them. An agent working on Phase 12 benefits from every mistake made in Phase 1. A new project benefits from cross-project knowledge. This is the flywheel.

What ONYX Is Not

- **Not an AI agent.** ONYX doesn't do the work. It dispatches agents that do the work, within a structured lifecycle.
- **Not a project management tool.** It doesn't have boards, sprints, or velocity charts. It has phases, states, and logs.
- **Not an operating system.** It doesn't manage processes, memory, or hardware. It manages execution flow through markdown.
- **Not a framework you import.** It's a CLI + vault convention. There's no SDK, no API to integrate against, no runtime dependency.

What ONYX Is

- A **controller loop** that reads vault state and dispatches work.
- A **vault convention** that gives every project the same observable structure.
- A **profile system** that lets different domains (engineering, content, research) specialize without breaking the shared model.
- A **knowledge compounder** that extracts durable learnings from every completed phase.

Architecture

ONYX has three layers. Each has a clear boundary and a clear job.

```
+-----------------------------------------------------------------+
|                          ONYX Core                              |
```

```
|  The nervous system. Shared by everything.                          |
|  Phase lifecycle, logging, knowledge, discovery, healing.           |
+---------------------------------------------------------------------+
         |                    |                    |
+------------------+ +------------------+ +------------------+
|   Engineering    | |     Content      | |    Research      |
|    Profile       | |     Profile      | |    Profile       |
|                  | |                  | |                  |
| repo_path        | | voice_profile    | | research_question|
| test_command     | | pipeline_stage   | | source_constraints|
| stack            | | safety_rules     | | synthesis_reqs   |
| definition_of_done| | distribution    | | output_format    |
| rollback_notes   | | episode_scope    | | confidence_gaps  |
+------------------+ +------------------+ +------------------+
         |                    |                    |
+------------------+ +------------------+ +------------------+
|    Almani        | |   ManiPlus       | |  Market Intel    |
|    KrakenBot     | |   Future podcast | |  Due Diligence   |
|    Pandora       | |   Content brand  | |  Literature Rev  |
|    (bundles)     | |   (bundles)      | |  (bundles)       |
+------------------+ +------------------+ +------------------+
```

Layer 1: ONYX Core — The Nervous System

The core is everything that every project shares. It never changes based on domain. It is the execution model itself.

What the core owns:

Concept	What it is	Why it's core
Project	A named unit of work with a goal, scope, and knowledge base	Every domain has projects
Phase	The smallest reviewable execution unit with explicit goals, tasks, acceptance criteria, and a linked log	Every domain decomposes into phases
Log	Append-only timestamped execution record per phase	Every domain needs audit trails
Knowledge	Accumulated learnings, decisions, gotchas from completed phases	Every domain benefits from compounding knowledge
Kanban	Read-only phase state view	Every domain needs status at a glance

Concept	What it is	Why it's core
Agent Log Hub	Index of all execution logs	Every domain needs log navigation

Core frontmatter keys (present on every phase, regardless of profile):

```yaml
project_id: "ProjectName"
phase_number: 1
phase_name: "Phase description"
state: ready                    # backlog | planning | ready | active |
blocked | completed
milestone: "v1.0"
phase_type: slice               # slice | spike | maintenance
risk: medium                    # low | medium | high
depends_on: [1, 2]
locked_by: ""
locked_at: ""
replan_count: 0
tags: [onyx-phase, phase-ready]
created: 2026-04-12
```

Core lifecycle states (the FSM):

```
backlog → planning → ready → active → completed
                                  ↘ blocked → planning
```

These six states are sacred. No profile can add states, remove states, or change the meaning of a state. A phase in `active` always means "an agent holds the lock and is executing." A phase in `blocked` always means "human intervention required." This is true for engineering. True for content. True for research. True for everything.

Core execution flow:

```
read vault → heal → discover → route → act → write vault → consolidate →
repeat
```

This loop is the heartbeat. Every `onyx run` invocation executes this loop. The profile affects what happens inside "act" (what context the agent receives, what verification means). The loop itself is invariant.

Layer 2: Profiles — Specialized Regions

A profile is a named set of extensions to the core. It adds:

1. **Extra frontmatter keys** that are valid on phase notes and Overview notes
2. **Extra doc templates** generated by `onyx init`
3. **A verification strategy** used by the executor to check acceptance
4. **Context injection rules** that determine what vault context the agent receives

A profile does NOT:

- Define new states
- Change the FSM transitions
- Alter the phase definition
- Modify the log format
- Override the core execution flow

How a profile is selected: One field in the Overview frontmatter.

```
# In the Overview note
---
project_id: ManiPlus
profile: content
---
```

That's the entire mechanism. The controller reads `profile`, loads the corresponding profile definition, and merges its schema/templates/verification with the core.

Engineering Profile

For software projects. Code lives in a repo. Verification means tests pass.

Extra Overview fields:

```
repo_path: /absolute/path/to/repo
stack: "Next.js 15, Prisma, PostgreSQL"
```

Extra Overview sections:

- `## Key Areas` — directory structure hints for agents
- `## Architecture Notes` — patterns agents must follow
- `## Agent Constraints` — hard rules agents must not violate

Extra phase fields:

```yaml
test_command: "npm test"
definition_of_done: "All tests pass, types check, no lint errors"
rollback_notes: "Revert commit abc123 if deploy fails"
```

Doc template: `{Project} - Repo Context.md` — stack, file structure, constraints, architecture.

Verification strategy: Run `test_command` after all tasks complete. Check exit code. If non-zero, phase is blocked.

Example bundles: Almani, KrakenBot, Pandora, OpenClaw Core.

Content Profile

For content production pipelines. Output is media (audio, video, text). Verification means safety and quality checks pass.

Extra Overview fields:

```yaml
voice_profile: "vault/knowledge.yaml"     # pointer to tone/style definition
safety_rules: "vault/knowledge.yaml"       # pointer to content safety
constraints
distribution_targets: [rss, youtube, tiktok, instagram]
```

Extra Overview sections:

- `## Content Pillars` — thematic lanes the content stays within
- `## Voice & Tone` — how the brand sounds (or pointer to voice profile file)
- `## Safety Constraints` — what must never appear in output

Extra phase fields:

```yaml
pipeline_stage: audio              # research | script | audio | video |
publish | engage | learn
episode_scope: 8                   # episode number, or "all" for cross-
episode work
safety_review_required: true
```

Doc template: `{Project} - Source Context.md` — voice profile, content pillars, safety rules, distribution targets, audience.

Verification strategy: Run safety filter on output. Check voice consistency. Verify distribution manifest is complete. Human review flag for sensitive content.

Example bundles: ManiPlus (podcast automation), future content brands.

Research Profile

For investigation, synthesis, and analysis work. Output is a memo or report. Verification means sources are cited and confidence is declared.

Extra Overview fields:

```
research_question: "What is the competitive landscape for X?"
source_constraints: "Peer-reviewed only, published after 2024"
output_format: memo                # memo | report | brief | annotated-
bibliography
```

Extra Overview sections:

- `## Research Question` — the driving question
- `## Source Constraints` — what counts as valid evidence
- `## Synthesis Requirements` — what the final output must contain

Extra phase fields:

```
source_count: 0                    # tracked: how many sources cited so far
confidence_level: low              # low | medium | high — self-assessed by
agent
confidence_gaps: []                # what's still unknown
```

Doc template: `{Project} - Research Brief.md` — question, constraints, methodology, source log.

Verification strategy: Check that `source_count > 0`. Check that `confidence_gaps` is populated (agent must declare what it doesn't know). Check that output matches `output_format` structure.

Example bundles: Market intelligence, due diligence, literature reviews, competitive analysis.

Operations Profile

For monitoring, maintenance, and operational tasks. Output is status reports and corrective actions. Verification means runbooks were followed.

Extra Overview fields:

```yaml
monitored_systems: [api, database, cdn]
runbook_path: "vault/runbooks/"
escalation_rules: "Page on-call if P1 unresolved after 15 min"
```

Extra phase fields:

```yaml
incident_id: ""                    # linked incident if reactive
severity: low                      # low | medium | high | critical
runbook_followed: false            # set true by agent after executing
runbook steps
```

Doc template: `{Project} - Operations Context.md` — systems, runbooks, escalation paths, SLAs.

Verification strategy: Check `runbook_followed == true`. Check that incident notes are populated if `incident_id` is set.

Example bundles: Infrastructure monitoring, incident response, platform health.

Creating Custom Profiles

Profiles are not hard-coded. A profile is defined by:

1. A name (string, used in `profile:` frontmatter)
2. A set of extra frontmatter keys (Zod schema extension)
3. A doc template (markdown file in `08 - System/Profiles/{ProfileName}/`)
4. A verification function (registered in the executor)

To create a new profile:

- Add a directory under `08 - System/Profiles/{ProfileName}/`
- Add a `profile.md` describing the extra fields and their purpose
- Add any doc templates the profile needs
- Register the schema extension and verification function in code

The core doesn't need to know about your profile in advance. It just reads the `profile` field and looks up the matching extension.

Layer 3: Bundles — Instantiated Functional Structures

A bundle is a project. It's what you get when you run `onyx init "My Project" --profile engineering`. It's the concrete vault folder with all its notes, phases, logs, and docs.

Every bundle contains (from core):

```
{Area}/My Project/
├── My Project - Overview.md          ← source of truth (includes profile:
field)
├── My Project - Knowledge.md         ← compounding learnings
├── My Project - Kanban.md            ← phase state board
├── My Project - Agent Log Hub.md     ← log index
├── Phases/
│   └── (phase notes created by planner)
├── Logs/
│   └── (log notes created by executor)
└── Docs/
    └── (profile-specific context docs)
```

Plus profile-specific docs:

Profile	Extra doc in Docs/
Engineering	`{Project} - Repo Context.md`
Content	`{Project} - Source Context.md`
Research	`{Project} - Research Brief.md`
Operations	`{Project} - Operations Context.md`

A bundle is self-contained. You can move it between vault areas, archive it, or share it. All its state travels with it.

The Sacred Phase Definition

This is the load-bearing abstraction of the entire system. It does not change across profiles, domains, or use cases.

> **A phase is the smallest reviewable execution unit with explicit goals, tasks, acceptance criteria, and a linked log.**

Let's unpack each word:

- **Smallest reviewable** — A phase should be completable in one agent session (typically 10-30 minutes of agent work). If it's bigger, decompose further.
- **Execution unit** — It's not a plan. It's not a wish. It produces a concrete, verifiable change.

- **Explicit goals** — The `## Summary` section states what this phase builds and why.
- **Tasks** — Atomic, checkable items. Each has files, steps, validation, and definition of done.
- **Acceptance criteria** — Binary conditions. Either all boxes are ticked or they're not.
- **Linked log** — Every phase has a corresponding `L{n}` log note with timestamped execution events.

What a phase looks like for different profiles:

Profile	Phase example	Tasks example	Acceptance example
Engineering	"Wire JWT authentication"	Create middleware file, add route guards, write tests	GET /api/me returns 200 with token, 401 without
Content	"Generate Episode 8 script"	Fetch research brief, generate outline, draft script, run voice alignment, run safety review	Script passes safety filter, matches voice profile, cites 3+ sources
Research	"Synthesize competitive landscape"	Gather sources, extract key findings, identify gaps, draft memo	Memo follows template, 10+ sources cited, confidence gaps declared
Operations	"Investigate API latency spike"	Pull metrics, identify root cause, draft remediation, execute runbook	Root cause documented, runbook followed, latency returned to baseline

Same structure. Different content. That's the whole idea.

How ManiPlus Works Under ONYX — A Complete Walkthrough

ManiPlus is a content automation agent for "The Beating Edge with Mani+" podcast. It's the canonical example of a non-engineering bundle running on ONYX.

The Overview

```
# ManiPlus - Overview.md
---
project_id: ManiPlus
profile: content
type: overview
status: active
```

```
voice_profile: "vault/knowledge.yaml"
safety_rules: "vault/knowledge.yaml"
distribution_targets: [rss, youtube, tiktok, instagram, shorts]
created: 2026-03-01
---

# ManiPlus — The Beating Edge with Mani+

## Scope
End-to-end content operating system for a healthcare podcast...

## Content Pillars
- Heart transplant recovery and life after
- Dialysis patient advocacy
- Healthcare innovation and research
- Resilience and faith in chronic illness

## Voice & Tone
- Human, sincere, faith-aware
- Opening: "Assalamu alaikum"
- Evidence-based when discussing medicine
- No personal medical advice

## Safety Constraints
- No unverified statistics
- No personalized medical advice
- Safety filters active in research, scripts, and engagement
```

The Phases (Exactly as They Exist Today)

P1 through P7 are completed. P8 is ready. The phase notes look the same as engineering phases — because phases are core. The only difference is the extra `pipeline_stage` field:

```
# P8 - Analytics feedback learning loop.md
---
project_id: ManiPlus
phase_number: 8
phase_name: Analytics feedback learning loop
state: ready
pipeline_stage: learn
episode_scope: all
safety_review_required: false
risk: medium
depends_on: [7]
```

```
tags: [onyx-phase, phase-ready]
---
```

The Execution Flow

When `onyx run --project ManiPlus` fires:

1. **Core discovers** P8 as `phase-ready` with all dependencies met
2. **Core routes** P8 to `execute`
3. **Core acquires lock** — sets `locked_by`, `locked_at`, `phase-active`
4. **Profile injects context** — because `profile: content`, the executor loads `Source Context.md` instead of `Repo Context.md`, injects voice profile and safety rules into the agent prompt
5. **Core runs task loop** — exactly the same as engineering: pick task, classify complexity, spawn agent, tick checkbox, log event
6. **Profile verifies** — instead of running `npm test`, the content verification checks safety filter pass and voice consistency
7. **Core consolidates** — extracts learnings to `ManiPlus - Knowledge.md`
8. **Core releases lock** — `phase-completed`

The core loop is identical. The profile only affects steps 4 and 6.

The Source Context Doc

Instead of `Repo Context.md` (which describes stack, file structure, architecture), ManiPlus has:

```
# ManiPlus - Source Context.md

## Voice Profile
- Tone: human, sincere, faith-aware
- Opening greeting: "Assalamu alaikum"
- Style: patient-first storytelling; evidence-based medicine; no clinical
coldness

## Safety Constraints
- No personalized medical advice
- No unverified statistics
- Safety filters active in: research briefs, script generation, engagement
replies

## Content Pillars
- Heart transplant recovery
```

```
- Dialysis patient advocacy
- Healthcare innovation
- Resilience and faith

## Distribution Targets
- RSS (Spotify for Podcasters)
- YouTube (full episodes, 1920x1080)
- TikTok (shorts, 1080x1920)
- Instagram Reels
- YouTube Shorts

## Pipeline Stages
research → script → audio → video → publish → engage → learn

## Medical Pronunciation Dictionary
12 terms with phonemes for ElevenLabs TTS:
hemodialysis, immunosuppressant, nephrology, dialysis, erythropoietin,
peritoneal, tacrolimus, glomerulonephritis, creatinine, arteriovenous,
mycophenolate, calcineurin
```

This is the content profile's equivalent of "here's the stack and the file structure." It tells the agent everything it needs to produce safe, on-brand content.

The Agent Ecosystem — How Specialized Agents Feed Back Through the Vault

This is the most powerful pattern in ONYX. It's not a feature — it's an emergent property of vault-as-truth.

The Pattern

Every project bundle has a Knowledge note. Agents write to it. Other agents read it. The vault is the message bus.

```
┌──────────────────┐      ┌──────────────────┐      ┌──────────────────┐
│   ManiPlus       │      │    Almani        │      │   KrakenBot      │
│   Agent          │      │    Agent         │      │   Agent          │
│ (content)        │      │ (engineering)    │      │ (engineering)    │
└──────────────────┘      └──────────────────┘      └──────────────────┘
         │                         │                         │
         │ writes                  │ writes                  │ writes
         ▼                         ▼                         ▼
   Knowledge.md              Knowledge.md              Knowledge.md
   Source Context             Repo Context              Repo Context
```

```
    Episode Feed            Decisions.md           Decisions.md
          │                       │                       │
          └───────────────────────┴───────────────────────┘
                              │
                              ▼
        ┌─────────────────────────┐
        │   Cross-Project         │
        │   Knowledge.md          │
        │   (08 - System/)        │
        └─────────────────────────┘
                              │
                              ▼
        ┌─────────────────────────┐
        │   OpenClaw Agent        │
        │   (reads everything     │
        │    via vault glob)      │
        └─────────────────────────┘
```

How It Works in Practice

Step 1: Specialized agent does work.
The ManiPlus agent runs P8 (analytics loop). It discovers that episodes about dialysis get 3x more engagement than transplant episodes. It writes this to `ManiPlus - Knowledge.md` under `## Learnings`.

Step 2: Knowledge consolidation.
The ONYX consolidator extracts broadly applicable learnings to `08 - System/Cross-Project Knowledge.md`. The dialysis insight stays project-local. But the learning "audience engagement correlates with emotional resonance, not medical complexity" is broadly applicable — it goes to cross-project knowledge.

Step 3: Another agent benefits.
When Almani's agent plans a new feature (health-related content recommendations), it reads cross-project knowledge and discovers the emotional resonance finding. It factors this into its recommendation algorithm design.

Step 4: The OpenClaw orchestrator sees everything.
The top-level OpenClaw agent reads all Knowledge notes across all bundles. It can surface cross-project opportunities: "ManiPlus has audience data that could inform Almani's content strategy."

Future: Persistent Specialized Agents

As projects mature, they can spawn **persistent agents** that run continuously (via cron or daemon) rather than phase-by-phase:

Project	Persistent Agent	What It Does	Writes To
KrakenBot	Trading Monitor	Watches positions, logs P&L, flags anomalies	`KrakenBot – Knowledge.md`
ManiPlus	Engagement Monitor	Watches YouTube comments, suggests replies	`ManiPlus – Knowledge.md`
Almani	Deploy Monitor	Watches Railway deploys, logs errors	`Almani – Knowledge.md`
Platform	Health Monitor	Watches all project health, surfaces blockers	`Cross-Project Knowledge.md`

These agents don't need new infrastructure. They're just cron jobs that read vault context, do their specialized work, and write findings back to Knowledge notes. The vault is their shared memory.

Use Cases

Software Engineering (Primary, Today)

Bundle: Engineering profile. Repo Context doc. Test-based verification.

Examples: Almani (SaaS), KrakenBot (trading bot), Pandora (enterprise platform), OpenClaw Core.

Flow: Overview defines scope and stack. Planner decomposes into phases. Atomiser generates tasks with file paths, steps, validation. Agent writes code, runs tests. Consolidator extracts learnings. Knowledge compounds.

What ONYX adds over raw agent usage: Structure, safety (locks prevent collisions), compounding knowledge, audit trail, multi-project coordination.

Content Production

Bundle: Content profile. Source Context doc. Safety-based verification.

Examples: ManiPlus (podcast), future content brands.

Flow: Overview defines voice, safety rules, distribution. Planner decomposes into pipeline stages (research → script → audio → video → publish). Agent produces content within safety constraints. Verification checks voice consistency and safety filter pass. Knowledge captures audience insights.

What ONYX adds: Consistent pipeline across episodes, safety enforcement, voice drift prevention, audience learning loop.

Research & Analysis

Bundle: Research profile. Research Brief doc. Citation-based verification.

Examples: Market intelligence, competitive analysis, due diligence, literature reviews.

Flow: Overview defines research question and source constraints. Planner decomposes into research phases (gather → analyze → synthesize → report). Agent researches within constraints, declares confidence level and gaps. Verification checks source count and gap declaration. Knowledge captures methodology insights.

What ONYX adds: Structured synthesis, confidence tracking, source accountability, methodology improvement over time.

Operations & Monitoring

Bundle: Operations profile. Operations Context doc. Runbook-based verification.

Examples: Infrastructure monitoring, incident response, platform health checks.

Flow: Overview defines monitored systems and escalation rules. Phases can be reactive (incident response) or proactive (health checks). Agent follows runbooks, documents findings. Verification checks runbook adherence. Knowledge captures operational patterns.

What ONYX adds: Runbook enforcement, incident documentation, pattern detection across incidents.

Personal Knowledge Work

Bundle: Research or custom profile. Lightweight verification.

Examples: Book notes, course learning, skill development tracking.

Flow: Overview defines learning goal. Phases represent units of study. Agent assists with synthesis and connection-making. Knowledge captures insights and connections.

What ONYX adds: Structure for long-term learning projects, compounding insights, connection across topics.

Multi-Agent Coordination

Bundle: Any profile. Multiple agents working on different phases of the same project.

How it works: ONYX's lock system means two agents can't work on the same phase simultaneously. But they can work on different phases of the same project, or different projects entirely, in parallel. The vault state machine prevents collisions. Knowledge notes serve as the shared context layer.

What ONYX adds: Safe parallelism without explicit coordination protocols. The vault IS the coordination protocol.

The Interface Layer — ONYX as the Bridge Between Context and Agents

This is the key mental model. ONYX is not an agent. It's not a project manager. It's an **interface layer** between two things:

1. **Project context** — goals, scope, knowledge, constraints, history (lives in the vault)
2. **Specialized agents** — AI models with tools that do actual work (live in code)

```
                ┌──────────────────────────────┐
                │        YOUR VAULT            │
                │                              │
                │  Overview    Knowledge        │
                │  Phases     Logs              │
                │  Docs       Decisions         │
                │                              │
                └──────────────────────────────┘
                           │
                           │
                    reads & writes
                           │
                ┌──────────┴───────────────────┐
                │         ONYX CORE            │
                │                              │
                │  discover → route → act      │
                │  heal → consolidate → learn   │
                │                              │
                │  + Profile (context shaping)  │
                │  + Verification (quality gate) │
                │                              │
                └──────────────────────────────┘
                           │
                           │
                      dispatches
                           │
            ┌──────────────┴──────────────────┐
            │                                 │
```

```
        │              │              │
        ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
        │  Claude      │  │  Cursor      │  │  Custom      │
        │  Code        │  │  Agent       │  │  Agent       │
        │              │  │              │  │              │
        └──────────────┘  └──────────────┘  └──────────────┘
```

Without ONYX, you're manually briefing agents every time. You're copy-pasting context. You're losing learnings between sessions. You're running multiple projects with no shared structure.

With ONYX, the vault holds the context. The profile shapes how that context is delivered. The core ensures execution is safe, logged, and learning-producing. Agents just do their specialized work.

That's the utility: ONYX is the thinnest possible layer that turns a pile of vault notes and a capable agent into a reliable, compounding execution system.

Configuration

The Config File

```json
// onyx.config.json
{
  "vault_root": "/home/jamal/Obsidian/GroundZeroVault",
  "agent_driver": "claude-code",
  "llm": {
    "model": "anthropic/claude-sonnet-4-6"
  },
  "model_tiers": {
    "planning": "anthropic/claude-opus-4-6",
    "light": "anthropic/claude-haiku-4-5-20251001",
    "standard": "anthropic/claude-sonnet-4-6",
    "heavy": "anthropic/claude-opus-4-6"
  },
  "max_iterations": 20,
  "stale_lock_threshold_ms": 300000,
  "projects_glob": "{02 - Fanvue/**,03 - Ventures/**,05 - Finance/**,10 -
OpenClaw/**}",
  "repos_root": "/home/jamal/clawd/openclaw/projects",
  "profiles_dir": "08 - System/Profiles",
  "notify": {
    "stdout": true,
    "whatsapp": { "target": "+447..." }
```

```
  }
}
```

Environment Variables

```
ONYX_VAULT_ROOT=/home/jamal/Obsidian/GroundZeroVault
OPENROUTER_API_KEY=sk-or-...
AGENT_DRIVER=claude-code
```

CLI Commands

Command	What it does
`onyx run`	Full autonomous loop: heal, discover, route, execute, consolidate
`onyx run --once`	Single iteration then exit (for cron)
`onyx run --project <name>`	Scope to one project
`onyx run --dry-run`	Preview without executing
`onyx plan <project>`	Create phases from Overview or atomise backlog phases
`onyx plan <project> --extend`	Add new phases based on current scope
`onyx init <name> --profile <p>`	Create new project bundle with profile-specific templates
`onyx status`	Show all projects, phases, states
`onyx heal`	Fix stale locks, drift, broken links
`onyx doctor`	Pre-flight check: vault, keys, agent, deps
`onyx reset <phase>`	Reset blocked/active phase to ready
`onyx logs [phase]`	Show execution logs
`onyx consolidate`	Archive completed phases, extract learnings
`onyx import <linearId>`	Import Linear project as vault bundle
`onyx capture <text>`	Quick-capture to Inbox
`onyx dashboard`	Launch web dashboard

The Three Laws (Unchanged from Core)

These are absolute. No profile, bundle, or agent may violate them.

Law 1 — Vault is the only state.
No sidecar files. No external databases. No in-memory state that doesn't reflect from vault frontmatter. Read the vault to know what's true.

Law 2 — Agents write to exactly two files per phase.
The phase note (tick checkboxes, set tag, set lock) and the log note (append events). Nothing else. Agents must not touch Overview, Knowledge, Kanban, or other phases.

Law 3 — Tags are the FSM, locks are in frontmatter.
The `state:` field is canonical. The `phase-*` tag is kept for Obsidian queries. `locked_by` and `locked_at` in frontmatter are the lock. No external lock service.

Principles

1. **Phase is sacred.** The smallest reviewable execution unit. No profile redefines it.
2. **Vault is truth.** If it's not in the vault, it didn't happen.
3. **Profiles are thin.** Extra fields, extra templates, extra verification. Nothing more.
4. **Knowledge compounds.** Every completed phase teaches the next one.
5. **Agents are disposable.** Swap Claude for Cursor, swap Cursor for a custom agent. The vault doesn't care.
6. **Human in the loop.** Blocked phases surface requirements. The system asks for help instead of guessing.
7. **Observable by default.** Logs, Kanban, Knowledge — everything is visible in Obsidian without special tooling.
8. **Convention over configuration.** The vault structure IS the configuration. Name things right and the system finds them.
9. **Domain agnostic.** Engineering is not special. Content is not special. The execution model is universal.
10. **Least mechanism.** Add nothing unless it earns its complexity. A string field in frontmatter is better than a profile service. A markdown file is better than a database table.

Glossary

Term	Definition
ONYX	The orchestration layer for Obsidian. CLI + vault convention + controller loop.
Core	The shared execution model: phases, logs, knowledge, FSM, healing, discovery.
Profile	A named extension to the core: extra fields, templates, verification strategy.
Bundle	An instantiated project folder in the vault. Created by `onyx init`.
Phase	The smallest reviewable execution unit with goals, tasks, acceptance criteria, and a log.
Log note	Append-only timestamped record of phase execution events.
Knowledge note	Accumulated learnings, decisions, and gotchas from completed phases.
Source Context	Content profile's equivalent of Repo Context — voice, safety, distribution.
Research Brief	Research profile's equivalent of Repo Context — question, constraints, methodology.
Verification	Profile-specific quality gate: tests (engineering), safety (content), citations (research).
Consolidation	Post-phase learning extraction. Writes to Knowledge.md and Cross-Project Knowledge.md.
Healing	Automatic vault repair: stale locks, drift, broken links.
Controller loop	The `onyx run` heartbeat: heal → discover → route → act → consolidate → repeat.

Summary

ONYX is the thinnest possible layer between your Obsidian vault and autonomous agent execution. It gives every project — code, content, research, operations — the same reliable lifecycle: plan, decompose, execute, verify, learn. Profiles specialize without fragmenting. Knowledge compounds across phases and projects. The vault is the only truth, and everything is observable in Obsidian.

One nervous system. Many specializations. Every phase sacred.

```
ONYX Core = the nervous system
Profiles  = specialized regions
```

```
Bundles   = instantiated functional structures
```

That's ONYX.